PRODUCT REQUIREMENT DOCUMENT (PRD) & TECHNICAL SPECIFICATION
Project: Multilingual Conversational AI Agent for Restaurant Reservations (Pakistan Market)
Document Version: 1.0.0
Target Deployment Cost: $0 (Leveraging Free-Tier Infrastructure)

---

1. OBJECTIVE & EXECUTIVE SUMMARY
The goal of this project is to build an autonomous, conversational B2B AI Agent that can be white-labeled and sold to restaurant owners across Pakistan. The agent acts as an automated front-of-house receptionist capable of handling table availability checks, processing reservations, and answering basic FAQs via instant messaging channels (WhatsApp/Telegram).

A key competitive advantage of this agent is the complete elimination of language and literacy barriers within the localized market. It natively understands English, formal Urdu, Roman Urdu, and regional Punjabi dialects, processing intents accurately while maintaining strict brand and cultural guardrails.

---

2. CORE SYSTEM REQUIREMENTS

2.1 Multilingual Linguistic Routing Matrix
* **Inbound Capability:** The agent must parse and accurately extract entities (Name, Date, Time, Guest Count) from inputs sent in:
  * English
  * Formal Urdu (Arabic Script)
  * Roman Urdu (Urdu spoken using Latin/English characters)
  * Punjabi (Both text and common phonetic variations)
* **Outbound Enforcement (Crucial Constraint):** * If the user initiates communication in **English**, the agent responds in **English**.
  * If the user initiates in **Urdu** or **Roman Urdu**, the agent responds in **Urdu** or **Roman Urdu** respectively.
  * If the user initiates in **Punjabi**, the agent **must never** reply in Punjabi. It must gracefully interpret the request, but format its response using warm, polite, and culturally respectful **Urdu**.

2.2 Functional Operations
* **Real-Time Availability Inquiries:** Customers can query if a specific number of seats are available at a given date and time.
* **Automated Reservation Booking:** The agent handles end-to-end booking details, records them to the centralized database, and issues a structured confirmation token.
* **Strict Determinism via Tool Use:** The agent is explicitly banned from making up ("hallucinating") table openings. It must always query the live database to confirm capacity before telling a customer a table is free.

---

3. ARCHITECTURE & ZERO-COST TECH STACK

To allow for zero-cost deployment during the initial bootstrapping phase, the system architecture maps exclusively to robust, enterprise-grade free tiers:

* **LLM Engine:** Google AI Studio — Gemini 1.5 Flash / Gemini 2.5 Flash API. Provides a high-volume free tier per minute/day, excellent native comprehension of Urdu, Roman Urdu, and south-asian regional dialects, and native support for JSON schema enforcement and function calling.

* **Database Layer:** MongoDB Atlas (Shared Cluster M0 Free Tier). Provides 512MB of persistent storage, advanced aggregation pipelines for slot math, and native indexing—more than enough for early operational validation.
* **Application Hosting Layer:** Render / Vercel (Free Tiers). Deploys Node.js (Express) or Python (FastAPI) webhooks with automatic SSL termination.
* **Channel Layer:** Telegram Bot API (100% free) or WhatsApp Cloud API Sandbox (Free for developer testing with up to 5 verified recipient numbers).

---

4. MODULE ARCHITECTURE & IMPLEMENTATION TICKETS

MODULE 1: DATABASE LAYER & INVENTORY DEFINITION

[TICKET 1.1] MongoDB Atlas Cluster & Schema Configuration
* **Priority:** Critical
* **Description:** Initialize the MongoDB Atlas M0 free tier instance and construct the core transactional collections.
* **Database Models:**
  ```json
  // Collection 1: restaurants
  {
    "_id": "ObjectId",
    "name": "String",
    "total_capacity_seats": "Number",
    "operating_hours": {
      "open": "HH:MM",
      "close": "HH:MM"
    }
  }

  // Collection 2: reservations
  {
    "_id": "ObjectId",
    "restaurant_id": "ObjectId",
    "customer_name": "String",
    "phone_number": "String",
    "booking_date": "YYYY-MM-DD", // ISO Format
    "booking_time": "HH:MM",      // 24-Hour Format
    "guest_count": "Number",
    "status": "String"            // ["confirmed", "cancelled"]
  }
  ```
* **Database Constraints:** Build a compound index on `reservations` for `[restaurant_id, booking_date, booking_time]` to allow rapid mathematical aggregations of current occupancy.

MODULE 2: AGENT INTELLIGENCE & GUARDRAILS

[TICKET 2.1] Prompt Engineering & System Instructions
* **Priority:** High
* **Description:** Construct and inject the core system prompt inside the Gemini API initialization wrapper to strictly govern linguistic behavior.
* **Production System Text:**
  ```text
  ```

You are an elite, respectful, and highly efficient digital front-of-house reservation manager for [Restaurant Name] in Pakistan. Your goal is to accurately check seating availability and book reservations.

Strict Language Guardrails:
1. You are natively fluent in English, Urdu (Urdu Script), and Roman Urdu. Always match the primary language used by the guest.
2. BEHAVIOR FOR PUNJABI INPUT: If a guest messages you in Punjabi (e.g., "Veeray, 4 bandyan di thaan mil jasi aj rati 8 baje?"), you must instantly parse their intent, extract variables (guests: 4, time: 20:00, date: today), but you MUST respond exclusively in polite, warm Urdu (e.g., "Ji bilkul, main aap ke liye abhi check karta hoon..."). You are strictly forbidden from outputting text strings in Punjabi.
3. Cultural Tone: Always employ professional Urdu honorifics such as 'Ji', 'Aap', and 'Tashreef rakhein' to maximize hospitality. Keep replies concise and formatted for mobile screen readability.
```

##### [TICKET 2.2] Function Calling & Tool Declarations
* **Priority:** Critical
* **Description:** Register native function declarations with the Gemini API client so the model can securely interface with the live backend instead of guessing availability.
* **Exposed Tool Definitions:**
1. `checkAvailability(date, time, guestCount)`: Returns a boolean indicating if the restaurant can accommodate the group.
2. `createReservation(customerName, phoneNumber, date, time, guestCount)`: Commits a confirmed slot to the database and returns a confirmation payload.

---

#### MODULE 3: BACKEND VALIDATION FULFILLMENT

##### [TICKET 3.1] Capacity Allocation & Aggregation Controller
* **Priority:** High
* **Description:** Implement the backend function that processes incoming data parameters passed by the LLM Tool Call and evaluates true real-time inventory.
* **Mathematical Logic:**
$$\text{Sum of Guest Counts for Confirmed Bookings in Target Window} + \text{Requested Guest Count} \le \text{Total Restaurant Capacity}$$
* **Acceptance Criteria:**
* If capacity allows, return `{"available": true}` back to the LLM agent state loop.
* If capacity is exceeded, return `{"available": false, "suggested_alternative_slots": ["HH:MM"]}`.

---

#### MODULE 4: EXTERNAL AGENT GATEWAY

##### [TICKET 4.1] Webhook Receiver Integration
* **Priority:** Medium
* **Description:** Set up an Express or FastAPI endpoint `/webhook` to capture incoming payload requests from messaging platforms (Telegram/WhatsApp).
* **Data Pipeline:** Inbound Chat JSON -> Extract text string and User ID -> Fetch or create Chat Session History -> Send payload to Gemini Live Context -> Execute Tool Call (if triggered) -> Parse final textual response string -> Send outbound HTTP POST response back to messaging channel API.

---

### 5. ACCEPATANCE & TESTING PROTOCOL
```

Before launching the agent to restaurant clients, the following functional validation test cases must pass:

Test ID	Input Message (Test Case)	Expected Linguistic Behavior	Expected Functional Action
TC-001	"Hi, I want a table for two tonight at 8 PM."	Respond in English	Triggers `checkAvailability` tool
TC-002	"Assalam-o-Alaikum, kya aaj raat 9 baje 5 logon ki jagah hogi?"	Respond in Roman Urdu / Urdu	Triggers `checkAvailability` tool
TC-003	"O yaar, sanu kal dophar nu 2 baje 6 bandyan di table chahidi ae." *(Punjabi)*	**Must respond in polite Urdu.** (No Punjabi allowed).	Triggers `checkAvailability` tool for tomorrow at 14:00